

Desenvolver um aplicativo Android moderno segue o padrão de arquitetura MVVM (Model-View-ViewModel) com Unidirectional Data Flow (UDF). A ideia é ter a interface gráfica (UI / View) reagindo ao estado fornecido pelo ViewModel, que por sua vez gerencia as regras de negócio e dados.

Abaixo está o passo a passo completo para criar um projeto funcional (um contador simples com ViewModel) no Android Studio.

1.Criar um novo projeto no Android Studio:

1. Abra o Android Studio e clique em **New Project**.
2. Na aba **Phone and Tablet**, selecione o template **Empty Activity** (que usa Jetpack Compose por padrão).
3. Defina o nome do aplicativo (ex: MeuPrimeiroApp), o nome do pacote e certifique-se de que a linguagem seja **Kotlin**.
4. Clique em **Finish** e aguarde a sincronização do Gradle.

2.Adicionar as dependências no Gradle:Arquivo build.gradle.kts (Module :app).

Verifique se as dependências do ciclo de vida e do ViewModel para Compose estão presentes no seu arquivo build.gradle.kts do módulo app:

```
Kotlin
dependencies {
    // ViewModel e integração com Compose
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.0")

    // DEMAIS DEPENDÊNCIAS DO COMPOSE...
}
```

Após alterar, clique em **Sync Now** no topo do arquivo.

3.Criar a classe ViewModel:

Crie um novo arquivo Kotlin chamado ContadorViewModel.kt. O ViewModel deve manter o estado observável via StateFlow para garantir a reatividade da UI.

```
Kotlin
import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow

class ContadorViewModel : ViewModel() {

    // Estado privado que só pode ser alterado internamente
    private val _contador = MutableStateFlow(0)

    // Estado público exposto como leitura para a UI
    val contador: StateFlow<Int> = _contador.asStateFlow()

    fun incrementar() {
        _contador.value += 1
    }

    fun decrementar() {
        if (_contador.value > 0) {
            _contador.value -= 1
        }
    }
}
```

4.Criar as telas Composable (View):

Crie o arquivo ContadorScreen.kt. É uma boa prática separar o Composable que gerencia a lógica/ViewModel (Stateful) do Composable puramente visual (Stateless).

Kotlin

```
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import androidx.lifecycle.viewmodel.compose.viewModel
```

// Componente Stateful: Obtém o ViewModel e passa o estado

@Composable

```
fun ContadorRoute(
    viewModel: ContadorViewModel = viewModel()
){
    val valorContador by viewModel.contador.collectAsStateWithLifecycle()
```

ContadorScreen(

```
    valor = valorContador,
    onIncrementar = { viewModel.incrementar() },
    onDecrementar = { viewModel.decrementar() }
)
```

```
}
```

// Componente Stateless: Apenas desenha a tela com base no estado recebido

@Composable

fun ContadorScreen(

```
    valor: Int,
    onIncrementar: () -> Unit,
    onDecrementar: () -> Unit
)
```

```
{
```

Column(

```
    modifier = Modifier.fillMaxSize(),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center
)
```

```
{
```

```
    Text(text = "Contador", fontSize = 24.sp)
```

```
    Text(text = "$valor", fontSize = 48.sp)
```

```

        Spacer(modifier = Modifier.height(16.dp))

        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            Button(onClick = onDecrementar) {
                Text(text = "-")
            }
            Button(onClick = onIncrementar) {
                Text(text = "+")
            }
        }
    }
}

```

5.Vincular a tela na MainActivity:

No arquivo MainActivity.kt, chame a função Composable principal dentro do bloco setContent.

```

Kotlin
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.ui.Modifier
import com.exemplo.meuprimeiroapp.ui.theme.MeuPrimeiroAppTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MeuPrimeiroAppTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    ContadorRoute()
                }
            }
        }
    }
}

```

```
}  
}  
}  
}  
}
```

6.Executar o Aplicativo:

1. Selecione um emulador (AVD) ou conecte um dispositivo físico via USB/Wi-Fi.
2. Clique no botão **Run (Play verde)** no menu superior do Android Studio.
3. Ao interagir com os botões, o ViewModel atualizará o estado e o Jetpack Compose fará a **recomposição** automática apenas do texto do contador.